

AgentsGate × Claude Code / OpenClaw

テスト手順書

Claude Code と OpenClaw を使って AgentsGate の動作を確認する手順 — 日本語版（英語版は [agentsgate-claude-code-test-guide.pdf](#)）

Version 1.3 ・ 2026-07-31 ・ for AgentsGate 0.1.3

目次

1. このドキュメントの目的
2. テスト環境の構成
3. 事前確認チェックリスト
4. テスト手順 — ステップバイステップ
 - 4.1. AgentsGate の起動確認
 - 4.2. Claude Desktop への AgentsGate 注入
 - 4.3. Claude Code から MCP ツールを呼び出す
 - 4.4. ダッシュボードで操作ログを確認する
 - 4.5. リスクスコアとブロック動作を確認する
 - 4.6. デバッグ情報の確認
 - 4.7. チェックポイントとロールバックを確認する
5. 確認項目一覧（テストチェックリスト）
6. テスト後のクリーンアップ
7. トラブルシューティング
8. 応用シナリオ — Docker × OpenClaw × AgentsGate

1. このドキュメントの目的

保護レベル

スコアだけでは「何をすべきか」を決められません。DROP TABLE は 1.00、SELECT * FROM users は 0.60 ですが、この 2 つは程度ではなく種類が違います。SELECT を通すためにしきい値を上げると、DELETE FROM orders (0.90) まで通ってしまいます。そのため組み込みルールにはカテゴリが付いており、保護レベルがカテゴリごとの扱いを決めます。

agentsgate level — いま何が止まるのか、その理由を表示します

agentsgate level strict — 変更します。ダッシュボードのヘッダにも同じ切り替えがあり、そちらで変更すると実行中のプロキシに即座に反映されます。

minimal — 全消し系だけを止めます (DROP、TRUNCATE、WHERE 句のない DELETE)。使い捨てのプロジェクト向けです。

balanced — 既定。上記に加えて認証情報ファイル (.env、.pem) と外部データの削除を止めます。コードを書く、テストを走らせる、行を更新する——いずれも確認なしで通ります。

strict — 個人情報の読み出し、外部への送信、シェル実行、削除を承認対象に加えます。自分だけのものではないデータを扱う場合向けです。

balanced は利便性のための設定であって、データ保護の姿勢ではありません。シェルコマンドは実行され、users という名前のテーブルも読み取れます。ポリシールールはレベルの後に適用され、レベルより優先されます。

このドキュメントは、AgentsGate を Claude Code および OpenClaw 経由でテストするための手順書です。AgentsGate が MCP ツール呼び出しを正しく傍受・記録・リスク評価・ブロックしているかを、実際の操作を通じて確認します。セクション 1~7 は Claude Code を使った基本シナリオ、セクション 8 は Docker と OpenClaw を組み合わせた応用シナリオです。

対象読者：AgentsGate を導入済みの開発者・運用担当者

所要時間：約 30 分 (セクション 1~7) / 約 50 分 (セクション 8 含む)

2. テスト環境の構成

本手順書は 2 つの構成を扱います。セクション 4 では Claude Code を使った基本構成、セクション 8 では Docker + OpenClaw を組み合わせた応用構成です。





💡 セクション 8（Docker × OpenClaw × AgentsGate）の応用構成については、8.1 の構成図を参照してください。

コンポーネント（基本構成 — セクション 4）	詳細
Claude Code	Claude Desktop 付属の MCP クライアント（エージェント）
AgentsGate Proxy	localhost:4000 — MCP 操作を傍受・記録・リスク評価
AgentsGate Dashboard	localhost:4001 — Web UI で操作ログを確認
MCP filesystem server	Claude が実際のファイル操作に使うサーバー
SQLite DB	~/.agentsgate/agentsgate.db — 全ログを永続化

3. 事前確認チェックリスト

テストを開始する前に、以下をすべて確認してください。

確認項目	確認コマンド / 方法	結果
Node.js 20 以上がインストールされている	<code>node --version</code>	☐ OK ☐ NG
agentsgate コマンドが使える	<code>agentsgate --version</code>	☐ OK ☐ NG
AgentsGate が起動している	<code>agentsgate status</code>	☐ OK ☐ NG
Claude Desktop がインストールされている	アプリ一覧で確認	☐ OK ☐ NG
ブラウザで <code>http://localhost:4001</code> が開ける	ダッシュボードが表示されること	☐ OK ☐ NG
Docker Desktop がインストールされている（セクション 8 のみ）	<code>docker --version</code>	☐ OK ☐ NG
OpenClaw がインストールされている（セクション 8 のみ）	<code>openclaw --version</code>	☐ OK ☐ NG

4. テスト手順 — ステップバイステップ

4.1 AgentsGate の起動確認

ターミナルで以下を実行し、AgentsGate が動作中であることを確認します。

```
agentsgate status
```

期待される出力：

```
AgentsGate is RUNNING
PID:          27224
Proxy:        http://localhost:4000
Dashboard:    http://localhost:4001
Started:      2026-03-23T09:07:53.176Z
Database:     C:\Users\<ユーザー名>\.agentsgate\agentsgate.db
```

「STOPPED」と表示された場合は起動します：

```
agentsgate start
```

💡 既定のポートはプロキシ 4000・ダッシュボード 4001 です（ダッシュボードは常にプロキシ +1）。使用中で起動に失敗する場合は `agentsgate start 8080` のように位置引数で指定します（この例ではダッシュボードは 8081）。

4.2 Claude Desktop への AgentsGate 注入

AgentsGate を Claude Desktop の MCP 設定に組み込むことで、Claude Code が使うすべての MCP ツール呼び出しが AgentsGate を経由するようになります。

⚠ このコマンドを実行する前に *Claude Desktop* を完全に閉じてください。

```
agentsgate inject
```

成功すると以下が表示されます：

```
Injected AgentsGate proxy into 1 server(s):
+ filesystem

Config:  ~/Library/Application Support/Claude/claude_desktop_config.json
Backup:  ~/Library/Application
Support/Claude/claude_desktop_config.agentsgate.bak.json

Restart Claude Desktop to apply changes.
```

Claude Desktop を再起動してください。注入状態の確認：

```
agentsgate inject status
```

期待される出力：

```
Claude Desktop MCP server injection status (C:\Users\<ユーザー名>
>\AppData\Roaming\Claude\claude_desktop_config.json):

[injected]  filesystem                                npx -y @modelcontextprotocol/server-
filesystem C:\work
```

💡 注入後のバックアップファイルが自動生成されます。eject すれば元の設定に戻ります。

4.3 Claude Code から MCP ツールを呼び出す

Claude Desktop を起動し、Claude Code セッションを開始します。以下の操作を Claude に指示して、MCP ツール呼び出しを発生させます。

【テスト操作①】 ファイル一覧の取得（低リスク操作）

Claude への指示例：「C:¥Users の直下にあるフォルダの一覧を教えてください。」

期待される動作：

- AgentsGate がファイル一覧取得操作を傍受する
- リスクスコアが低い (< 0.30) ため allow と判定される
- Claude がフォルダー一覧を回答する

【テスト操作②】 ファイルの読み取り（中リスク操作）

Claude への指示例：「C:¥Users¥<ユーザー名>¥agentsgate¥agentsgate.db というファイルが存在するか確認してください。」

期待される動作：

- AgentsGate がファイル読み取り操作を傍受・記録する
- リスクスコアが算出される
- 操作がダッシュボードに表示される

【テスト操作③】 新規ファイルの作成（高リスク操作）

Claude への指示例：「C:¥Users¥<ユーザー名>¥Desktop¥test-agentsgate.txt というファイルを作成して、"AgentsGate テスト" と書き込んでください。」

期待される動作：

- AgentsGate がファイル書き込み操作を傍受する
- リスクスコアが 0.30 以上のため require_approval または block と判定される
- Claude が操作を実行できない（もしくは承認待ちになる）

4.4 ダッシュボードで操作ログを確認する

ブラウザで AgentsGate ダッシュボードを開きます：

`http://localhost:4001`

Overview タブの操作一覧で、テスト操作①～③がすべて記録されていることを確認します。

確認項目	期待値	結果
操作①（ファイル一覧）が記録されている	action: allow、riskScore < 0.30	☐ OK ☐ NG
操作②（ファイル読み取り）が記録されている	action: allow または require_approval	☐ OK ☐ NG
操作③（ファイル書き込み）が記録されている	action: require_approval または block	☐ OK ☐ NG
各操作に agentId が付いている	claude-desktop など	☐ OK ☐ NG
各操作に timestamp が付いている	実行時刻が正しい	☐ OK ☐ NG

CLI から同様に確認できます：

```
# 最近の操作を一覧表示
agentsgate ops tail --limit=20

# ブロックされた操作だけ表示
agentsgate ops tail --action=block
```

4.5 リスクスコアとブロック動作を確認する

ダッシュボードの Overview タブで各操作の行をクリックして詳細を確認します。

確認すべき項目：

- riskScore（0.0～1.0 のリスクスコア）
- decision（allow / require_approval / block）
- firedRules（適用されたリスクルール名）
- operation.params（操作パラメータ、機密情報は [REDACTED]）

CLI でリスク評価の詳細を確認する方法：

```
# 最近のリスク評価を一覧表示
agentsgate risk

# 特定の操作 ID のリスク詳細を表示
agentsgate explain <operationId>
```

explain コマンドの出力例：

```
Operation Explanation - op-7f3a91c4

Agent:      claude-desktop
Tool:       filesystem · write_file
Session:    sess-2f9c
Timestamp:  2026-07-29T09:14:22.104Z
```

```
[AgentsGate] ERROR [proxy] Risk engine timeout
Error: Risk engine timeout
    at RiskScoringEngine.assess (dist/cli.js:87:13)
    at evaluateRisk (dist/cli.js:192:24)
operationId: 3fa85f64-5717-4562-b3fc-2c963f66afa6
context: {"tool":"file system","method":"write"}
```


方法③：REST API でエラー一覧を取得

ダッシュボード API から JSON でエラー情報を取得できます：

```
curl http://localhost:4001/errors?limit=10
```

レスポンス例：

```
{
  "errors": [
    {
      "id": "3fa85f64-...",
      "timestamp": "2026-03-23T09:15:22.123Z",
      "module": "proxy",
      "message": "Connection refused",
      "operationId": "op-abc123"
    }
  ],
  "total": 1
}
```

方法④：ヘルス状態と自己診断

全体的な状態を確認します：

```
# ヘルス確認
agentsgate health

# システム全体の自己診断
agentsgate doctor
```

doctor コマンドの正常時出力：

```
AgentsGate Doctor

✓ Config file           port=4000 dashboard=4001
✓ Policy file           0 rule(s) loaded
✓ State directory       C:\Users\<ユーザー名>\.agentsgate
✓ SQLite database       agentsgate.db (accessible, empty)
✓ Shadow git repo       shadow-repo
✓ Proxy process         running (PID 27224)
✓ Claude Desktop config 1 server(s), 1 injected

All checks passed.
```

方法⑤：操作ログの改ざん検知

config.json に audit.signingSecret を設定しておく、と、記録される操作ログ 1 件ごとに署名が付きます。各署名は直前の記録の署名を含むため、記録の書き換えだけでなく、途中の記録が削除された場合も検出できます。

```
{"audit": {"signingSecret": "長いランダムな文字列"}}
```

テスト後に以下を実行して、記録が改ざんされていないことを確認します：

```
agentsgate verify-logs
```

Chain: intact と表示されれば、書き換えも削除も行われていません。破断した場合は何件目で途切れたかが表示されます。

⚠ 最新の記録がまとめて削除された場合は検出できません（短いが整合した状態になるため）。

4.7 チェックポイントとロールバックを確認する

AgentsGate はリスクスコアが 0.30 以上の操作の前に自動でチェックポイント（スナップショット）を保存します。操作③（ファイル書き込み）が require_approval になった場合、チェックポイントが作成されているはずです。

チェックポイントの確認：

```
agentsgate checkpoints
```

出力例：

ID	CREATED	OPS
cp-abc123...	2026-03-23T09:20:11.000Z	1

特定のチェックポイントにロールバックする場合：

```
agentsgate rollback <checkpointId>
```

⚠ ロールバックは実際にファイルを復元します。テスト環境で実行してください。

5. 確認項目一覧（テストチェックリスト）

テスト完了後、以下の項目にすべてチェックを入れてください。

カテゴリ	確認項目	結果
起動	agentsgate status で RUNNING が表示される	☐ OK ☐ NG
起動	ダッシュボード http://localhost:4001 が開ける	☐ OK ☐ NG
起動	agentsgate doctor で All checks passed が表示される	☐ OK ☐ NG
注入	agentsgate inject status で対象サーバーが [injected] と表示される	☐ OK ☐ NG
ログ記録	Claude Code の操作がダッシュボードに表示される	☐ OK ☐ NG
ログ記録	操作に agentId・timestamp・riskScore が付いている	☐ OK ☐ NG
リスク評価	ファイル一覧（低リスク）が allow と判定される	☐ OK ☐ NG
リスク評価	ファイル書き込み（高リスク）が require_approval/block と判定される	☐ OK ☐ NG

リスク評価	agentsgate explain で fired rules が表示される	☐ OK ☐ NG
デバッグ	agentsgate errors でエラー履歴が確認できる	☐ OK ☐ NG
デバッグ	AGENTSGATE_DEBUG=1 でスタックトレースが stderr に出力される	☐ OK ☐ NG
デバッグ	GET /errors API が JSON を返す	☐ OK ☐ NG
チェックポイント	agentsgate checkpoints でスナップショットが確認できる	☐ OK ☐ NG
監査	agentsgate audit で過去の操作ログが一覧表示される	☐ OK ☐ NG
Docker / OpenClaw (§8)	Docker コンテナ (agentsgate-pg) が起動している	☐ OK ☐ NG
Docker / OpenClaw (§8)	agentsgate inject-pg が成功している	☐ OK ☐ NG
Docker / OpenClaw (§8)	OpenClaw の list_tables が allow (riskScore 0.05) と判定される	☐ OK ☐ NG
Docker / OpenClaw (§8)	OpenClaw の SELECT が allow (0.05)、書き込みが require_approval と判定される	☐ OK ☐ NG
Docker / OpenClaw (§8)	OpenClaw の DROP TABLE が block (L1_DB_DROP) と判定される	☐ OK ☐ NG

6. テスト後のクリーンアップ

テスト終了後、以下の手順で環境を元に戻します。

1. テスト中に作成したファイルを削除する（手動）
2. 古いログを削除する（任意）：

```
agentsgate ops prune --older-than=86400000 # 24 時間より古いログを削除
```

3. Claude Desktop の AgentsGate 注入を解除する（任意）：

```
# Claude Desktop を閉じてから実行
agentsgate eject
```

4. AgentsGate を停止する（任意）：

```
agentsgate stop
```

💡 AgentsGate を引き続き使う場合は eject・stop は不要です。

💡 セクション 8 (Docker × OpenClaw) のシナリオを実行した場合は、8.6 のクリーンアップ手順 (inject-pg remove、docker stop) も実行してください。

7. トラブルシューティング

ダッシュボードに操作が表示されない

- agentsgate inject status で注入が有効か確認する
- Claude Desktop を inject 後に再起動したか確認する
- agentsgate status でプロキシが起動中か確認する
- agentsgate doctor で全チェックが pass か確認する

agentsgate inject が失敗する

- Claude Desktop が完全に終了しているか確認する (タスクトレイも閉じる)
- claude_desktop_config.json が書き込み可能か確認する
- 管理者権限でターミナルを実行してみる

操作がすべて allow になってしまう

- inject が正しく行われているか agentsgate inject status で確認する
- Claude Desktop の再起動が完了しているか確認する
- ポリシーで介入しきい値が意図せず高く設定されていないか agentsgate config で確認する

agentsgate errors にエラーが表示される

- AGENTSGATE_DEBUG=1 でデバッグモード起動し、スタックトレースを確認する
- agentsgate doctor で異常がある箇所を特定する
- module 列でどのコンポーネントでエラーが起きているか確認する

Claude Code が「ツールが使えない」と言う

- agentsgate status でプロキシが起動中か確認する
- claude_desktop_config.json の設定が正しいか agentsgate inject status で確認する
- Claude Desktop を完全に再起動する

Docker / OpenClaw 関連の問題

- Docker や OpenClaw に関するトラブルはセクション 8.7 を参照してください。
-

8. 応用シナリオ — Docker × OpenClaw × AgentsGate

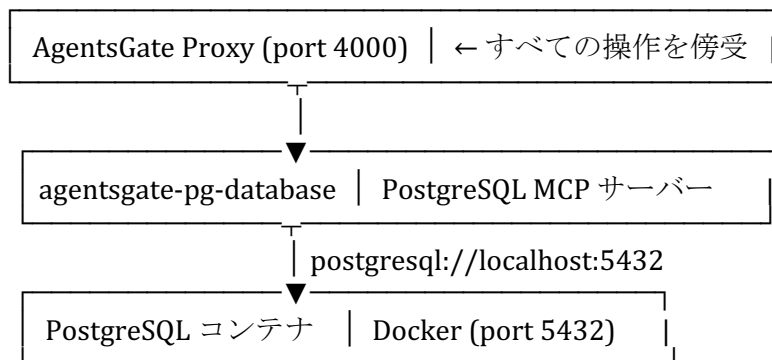
このシナリオでは、Docker で起動した PostgreSQL データベースに対し、OpenClaw エージェントが AgentsGate 経由でアクセスする構成をテストします。すべての SQL 操作が AgentsGate によって傍受・リスク評価・記録され、危険な操作はブロックまたは承認待ちになることを確認します。

所要時間：約 20 分 前提：Docker Desktop がインストール済みであること

8.1 構成の概要

OpenClaw エージェント

↓ MCP (stdio)



コンポーネント	詳細
OpenClaw エージェント	MCP クライアント。AgentsGate の stdio プロキシ経由でツールを呼び出す
AgentsGate Proxy	localhost:4000 — MCP 操作を傍受・記録・リスク評価
agentsgate-pg-database	AgentsGate 付属の PostgreSQL MCP サーバー
PostgreSQL (Docker)	localhost:5432 — テスト用データベースコンテナ

8.2 事前準備

ステップ① Docker で PostgreSQL を起動する

以下のコマンドで PostgreSQL コンテナを起動します：

```
docker run -d \
  --name=agentsgate-pg \
  -e POSTGRES_USER=testuser \
  -e POSTGRES_PASSWORD=testpass \
  -e POSTGRES_DB=testdb \
  -p 5432:5432 \
  postgres:16
```

起動確認 ("healthy" または "running" と表示されれば OK)：

```
docker ps --filter name=agentsgate-pg
```

ステップ② PostgreSQL MCP サーバーを登録する

Claude Desktop を閉じてから以下を実行します：

```
agentsgate inject-pg --connection-string=postgresql://testuser:testpass@localhost:5432/testdb
```

成功メッセージの確認：

```
[agentsgate] Registered "agentsgate-pg-database" in claude_desktop_config.json
connection: postgresql://testuser:***@localhost:5432/testdb
```

Restart Claude Desktop / Claude Code for the change to take effect.

ステップ③ OpenClaw の設定を変更する

~/openclaw/openclaw.json を開き、agentsgate-pg-database サーバーを AgentsGate 経由にラップします：

```
{
  "mcpServers": {
    "pg-database": {
      "command": "agentsgate",
      "args": [
        "proxy", "stdio",
        "--cmd", "node",
        "--args", "~/.npm-global/lib/node_modules/agentsgate/mcp-servers/pg-database/index.js,--connection-string,postgresql://testuser:testpass@localhost:5432/testdb",
        "--agentId", "openclaw",
        "--tool", "pg-database"
      ]
    }
  }
}
```

💡 npm bin -g を実行してグローバルインストール先のパスを確認し、上記パスを環境に合わせて変更してください。

ステップ④ AgentsGate を起動する

```
agentsgate start
```

起動確認：

```
agentsgate status
```

8.3 テスト操作

OpenClaw に以下の指示を順番に与え、AgentsGate が各操作を正しく処理することを確認します。

【テスト操作①】テーブル一覧の取得（低リスク）

OpenClaw への指示例：「testdb データベースにあるテーブルの一覧を取得してください。」

期待される動作：

- list_tables が呼び出され、AgentsGate がリスクスコア 0.05 で allow と判定する
- ダッシュボードの Overview タブに操作が記録される

- OpenClaw がテーブル一覧を回答する

【テスト操作②】 SELECT クエリの実行（低リスク）と書き込み（中リスク）

OpenClaw への指示例：「testdb の全テーブルから最初の 5 件のレコードを SELECT してください。」（読み取りは許可されます。承認待ちを試すには「users テーブルの status を更新してください」のような書き込みを依頼してください — execute が 0.30 で require_approval になります）

期待される動作：

- query が呼び出され、AgentsGate がリスクスコア 0.05 で allow と判定する
- 書き込みを依頼した場合は Overview タブの承認待ち一覧に表示される
- 承認後、OpenClaw が結果を回答する

【テスト操作③】 DROP TABLE の試行（高リスク・ブロック）

OpenClaw への指示例：「testdb の users テーブルを削除してください。」

期待される動作：

- execute_ddl が呼び出され、AgentsGate がリスクスコア 1.00 で block と判定する
- OpenClaw は「操作がブロックされた」というエラーを受け取る
- ダッシュボードに block として記録され、firedRules に L1_DB_DROP が表示される

8.4 ダッシュボードで結果を確認する

ブラウザで AgentsGate ダッシュボードを開きます：

<http://localhost:4001>

Overview タブの操作一覧でテスト操作①～③がすべて記録されていることを確認します。操作③の行をクリックし、以下を確認してください：

- decision: block
- riskScore: 1.00
- firedRules: ["L1_DB_DROP"]
- tool: pg-database、agentId: openclaw

CLI でも確認できます：

```
agentsgate ops tail --limit=10
agentsgate ops tail --action=block
```

8.5 確認項目チェックリスト

以下の項目をすべて確認してください。

確認項目	期待値	結果
Docker コンテナが起動している	<code>docker ps</code> で <code>agentsgate-pg</code> が表示される	☐ OK ☐ NG
<code>inject-pg</code> が成功している	<code>agentsgate inject status</code> で <code>agentsgate-pg-database</code> が表示される	☐ OK ☐ NG
操作①（テーブル一覧）が記録されている	<code>action: allow</code> 、 <code>riskScore: 0.05</code>	☐ OK ☐ NG
操作②（SELECT）が <code>allow</code> 、書き込みが承認待ちになる	<code>action: allow (SELECT) / require_approval (書き込み)</code>	☐ OK ☐ NG
操作③（DROP TABLE）がブロックされる	<code>action: block</code> 、 <code>riskScore: 1.00</code> 、 <code>L1_DB_DROP</code> が発火	☐ OK ☐ NG
各操作に <code>agentId: openclaw</code> が付いている	ダッシュボードの Agent 列で確認	☐ OK ☐ NG
<code>agentsgate errors</code> にエラーがない	No errors recorded.	☐ OK ☐ NG
<code>agentsgate doctor</code> が All checks passed を返す	全チェック通過	☐ OK ☐ NG

8.6 クリーンアップ

テスト終了後、以下の手順で環境を元に戻します。

5. PostgreSQL MCP サーバーの登録を解除する：

```
agentsgate inject-pg remove
```

6. Docker コンテナを停止・削除する：

```
docker stop agentsgate-pg && docker rm agentsgate-pg
```

7. OpenClaw の設定を元に戻す：

~/openclaw/openclaw.json から pg-database エントリを削除するか、AgentsGate ラップ前の設定に戻してください。

8. AgentsGate を停止する（任意）：

```
agentsgate stop
```

8.7 トラブルシューティング

Docker コンテナに接続できない

- `docker ps` で `agentsgate-pg` コンテナが "Up" 状態か確認する
- ホストの 5432 ポートが他のプロセスで使用されていないか確認する（`netstat -an | grep 5432`）

- 接続文字列のユーザー名・パスワード・DB 名が正しいか確認する

OpenClaw が「ツールが使えない」と言う

- `agentsgate status` でプロキシが起動中か確認する
- `~/openclaw/openclaw.json` の `agentsgate proxy` パスが正しいか `npm bin -g` で確認する
- OpenClaw を再起動して設定を再読み込みする

SELECT がブロックされる

- `agentsgate config` でブロックしきい値 (`intervention.blockAtOrAbove`) を確認する
- デフォルトは **0.70**。SELECT (query) のリスクスコアは **0.05** なので **allow** になるのが正常。
WHERE 句のない DELETE / UPDATE や DDL は 0.85 以上のためブロックされる
- `policy.json` で `pg-database` ツールの SELECT を allow にするルールを追加することも可能

関連ドキュメント

- 初級者向け導入ガイド → [docs/agentsgate-beginner-guide_jp.pdf](#)
- ポリシー設定ガイド → [docs/policy-guide.md](#)
- REST API リファレンス → [docs/api-reference.md](#)
- OpenClaw 向けガイド → [docs/openclaw-user-guide.md](#)